Experiment 5

Student Name: Krish Latawa
UID: 24BAI70541
Branch: CSE - AIML
Section/Group: 24AIT_KRG G1
Semester: 4
Date of Performance: 11/3/26
Subject Name: DBMS
Subject Code: 24CSH-298

Aim

To understand and apply conditional logic in SQL by using the modulus operator (MOD / %) to analyse numerical data and classify employee salaries as odd or even, thereby improving data analysis and decision-making skills in SQL.

Software Requirements

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

Objectives

- To write and execute SQL queries that use the MOD (%) operator to check employee salaries and display odd and even salary values separately from an employee table.

Practical/Experiment Steps

- Schema Architecture: Created the employees table with a structured format, ensuring the emp_id serves as the Primary Key for data integrity.
- Data Ingestion: Populated the table with diverse employee records including names and specific salary figures to provide a sufficient dataset for numerical analysis.

- Mathematical Operation: Leveraged the modulus operator (%) to perform a parity check on the salary column by dividing each value by 2.
- Conditional Classification: Integrated a CASE statement to evaluate the result of the modulus operation and assign a descriptive text label to each record.
- Dynamic Column Generation: Produced a virtual column named salary_type during execution to display the results of the logic without altering the physical table structure.

Procedure

- Initialised the database environment using PostgreSQL or Oracle XE and opened the query editor.
- Executed the CREATE TABLE script to define the employee information schema.
- Used INSERT INTO statements to add multiple rows of sample data, ensuring a mix of odd and even salary values.
- Drafted a SELECT query to retrieve all existing columns while preparing for the conditional logic implementation.
- Applied the % operator within a CASE structure to determine if the emp_sal remained with a zero or non-zero remainder.
- Assigned the labels 'Even Salary' or 'Odd Salary' based on the evaluation of the conditional branch.
- Ran the complete query to verify that numerical values (like 12000 vs. 15675) were accurately classified in the output window.
- Reviewed the resulting data grid to ensure the salary_type column correctly reflected the parity of the corresponding salary.
- Saved the final SQL script and captured screenshots of the results for record maintenance.

Input/Output Analysis

SQL Input Queries

```
CREATE TABLE workers(

    emp_id INT PRIMARY KEY,

    emp_name VARCHAR(20),

    emp_sal INT
```

```
);

INSERT INTO workers VALUES(10, 'Jackob', 12000);

INSERT INTO workers VALUES(6, 'Pritam', 15675);

INSERT INTO workers VALUES(7, 'Sharma', 13000);

INSERT INTO workers VALUES(8, 'Verma', 20001);

INSERT INTO workers VALUES(9, 'Tilak', 15000);

SELECT * ,

CASE

WHEN emp_sal%2 = 0 THEN 'Even Salary'

ELSE 'Odd Salary'

END AS salary_type

FROM workers;
```

Output

Data Output Messages Notifications

	emp_id [PK] integer	emp_name character varying (20)	emp_sal integer	salary_type text
1	10	Jackob	12000	Even Salary
2	6	Pritam	15675	Odd Salary
3	7	Sharma	13000	Even Salary
4	8	Verma	20001	Odd Salary
5	9	Tilak	15000	Even Salary

Learning Outcomes

- Gained proficiency in using the modulus operator to identify patterns in numerical data.
- Implemented CASE statements to categorise records based on mathematical results.
- Generated dynamic columns that provide analytical insights during data retrieval.
- Gained skills in transforming raw integers into meaningful descriptive categories for reporting.